

Resumen Procesos y Memoria

Tiempos de los procesos

- CPU (Tcpu): tiempo que efectivamente usa la CPU el proceso.
- Retorno (Tr): tiempo que transcurre entre que el proceso llega al sistema hasta que completa su ejecución.
- Espera (Te): tiempo que el proceso se encuentra en el sistema esperando, es decir el tiempo que pasa sin ejecutarse. $Te = (Tr - Tcpu)$
- Promedios (TPR y TPE): tiempos promedio de Retorno y Espera. Promedio calculado de los tiempos individuales de cada proceso del lote.

Planificación de CPU

La planificación (Scheduling) de CPU se ocupa del problema de decidir cual de los procesos que están en la cola de listos debe recibir la CPU.

Para ello existen diferentes tipos de algoritmos de Scheduling:

• Algoritmo FIFO (First Come First Served)

Cuando hay que elegir un proceso para ejecutarse, se selecciona el más antiguo. No favorece a ningún tipo de procesos, pero en principio se podría decir que los CPU-Bound terminan al comenzar su primera ráfaga, mientras que los I/O-Bound requieren múltiples ráfagas

JOB	LLEG.	CPU	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	TPR	TPE
1	0	6	>1	2	3	4	5	6<										6	0
2	1	3		>					1	2	3<							8	5
3	2	2			>							1	2<					9	7
4	3	4				>								1	2	3	4<	12	8

35 20

Adecuado para

- Sistemas por lotes (batch) donde los procesos no requieren interacción.
- Cuando no importa la rapidez de respuesta, sino la simplicidad.

Ventajas y desventajas

- Fácil de implementar.
- Es justo en el sentido de “primero en llegar, primero en servir”.
- No apropiativo, un proceso largo bloquea a los demás.
- Efecto convoy, los procesos cortos esperan mucho si hay uno largo antes.

• Algoritmo SJF (Shortest Job First)

Política non preemptive que selecciona el proceso con la ráfaga más corta.

Cálculo basado en la ejecución previa.

Procesos cortos se colocan delante de procesos largos (se va ordenando así la cola de listos).

Los procesos largos pueden sufrir starvation (inanición)

JOB	LLEG.	CPU	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	TPR	TPE
1	0	6	>1	2	3	4	5	6<										6	0
2	1	3		>							1	2	3<					10	7
3	2	2			>				1	2<								6	4
4	3	4				>								1	2	3	4<	12	8

34 19

Adecuado para

- Sistemas por lotes donde se conocen los tiempos de ejecución.
- No recomendable en sistemas interactivos o tiempo real.

Ventajas y desventajas

- Minimiza el tiempo promedio de espera.
- Puede causar inanición (procesos largos esperan indefinidamente).
- Requiere conocer o estimar el burst time.

• Algoritmo SRTF (Shortest Remaining Time First)

Versión apropiativa (preemptive) del SJF, selecciona el proceso con menos tiempo de ejecución es su siguiente rafaga.

Favorece a los procesos I/O-Bound

Si llega un proceso con menor tiempo de ejecución que el actual, este se adueña del CPU

JOB	LLEG.	CPU	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	TPR	TPE
1	0	6	>1										2	3	4	5	6<	15	9
2	1	3		>1			2	3<										5	2
3	2	2			>1	2<												2	0
4	3	4				>			1	2	3	4<						6	2
Queue: P1 P2 P3 P2 P4 P1																		28	13

Adecuado para

- Sistemas por lotes donde se conocen los tiempos de ejecución.
- No recomendable en sistemas interactivos o tiempo real.

Ventajas y desventajas

- Minimiza el tiempo promedio de espera.
- Puede causar inanición (procesos largos esperan indefinidamente).
- Requiere conocer o estimar el burst time.

• Algoritmo RR (Round Robin)

Cada proceso recibe un quantum (intervalo de tiempo fijo).

Cuando su tiempo se acaba, pasa al final de cola y se ejecuta el siguiente.

Ejemplo con Q=2

JOB	LLEG.	CPU	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	TPR	TPE
1	0	6	>1	2			3	4						5	6<			13	7
2	1	3		>	1	2							3<					10	7
3	2	2			>				1	2<								6	4
4	3	4				>					1	2				3	4<	12	8
Queue: P1 P2 P1 P3 P4 P2 P1 P4																		41	26

Adecuado para

- Sistemas interactivos o multitarea.
- Ideal cuando se busca equidad y respuesta rápida.

Ventajas y desventajas

- Justo con todos los procesos.
- Buen tiempo de respuesta.
- Si el quantum es muy grande, se comporta como FCFS.
- Si el quantum es muy pequeño hay muchas interrupciones (overhead).

• Algoritmo con uso de Prioridades

Cada proceso tiene un valor que representa su prioridad -> menor valor mayor prioridad. Se selecciona el proceso de menor prioridad que se encuentran en la Ready Queue.

Existe una Ready Queue por cada nivel de prioridad.

Puede ser apropiativo o no.

Ejemplo no apropiativo

JOB	LLEG	CPU	PRI	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	TPR	TPE
1	0	6	4	>1	2	3	4	5	6<										6	0
2	1	3	2		>									1	2	3<			12	9
3	2	2	3			>											1	2<	13	11
4	3	4	1				>			1	2	3	4<						7	3
Queue 1: P4																			38	23
Queue 2: P2																				
Queue 3: P3																				
Queue 4: P1																				

Ejemplo apropiativo

JOB	LLEG	CPU	PRI	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	TPR	TPE
1	0	6	4	>1										2	3	4	5	6<	14	8
2	1	3	2		>1	2					3<								7	4
3	2	2	3			>						1	2<						8	6
4	3	4	1				>1	2	3	4<									4	0
Queue 1: P4																			33	18
Queue 2: P2 P2																				
Queue 3: P3																				
Queue 4: P1 P1																				

Adecuado para

- Sistemas en tiempo real o críticos, donde algunos procesos son más urgentes.

Ventajas y desventajas

- Permite controlar qué procesos son más importantes.
- Flexible para sistemas con diferentes niveles de urgencia.
- Puede causar inanición de procesos de baja prioridad.
- Requiere gestión cuidadosa de prioridades.

• Algoritmo con uso de Recursos

Cada proceso puede tener uno o más recursos de entrada salida.

El instante del I/O es el instante del proceso, no de los instantes generales.

Cuando un proceso ejecuta un recurso los demás procesos pueden seguir ejecutándose.

Ejemplo FCFS con recursos

JO B	LLE G	CP U	I	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	TPR	TPE
1	0	6	(R1,2,1)	>1	2	R1			3	4	5	6<							9	3
2	1	3	(R2,2,4)		>	1	2	R2	R2						3<				11	8
3	2	2	(R2,1,1)			>		1	CR2	R2						2<			11	9
4	3	4	(R3,2,1)				>						1	2	R3		3	4<	12	8
Ready Queue: P1 P2 P3 P1 P4 P2 P3																			43	27
Queue R1: P2																				
Queue R2: P3																				
Queue R3: P1																				

Criterios de desempate de los Algoritmos de Planificación

- Orden de llegada del proceso
- PID del proceso

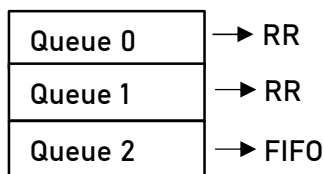
Siempre se mantiene la misma política.

• Esquema de colas multinivel

Es un método de planificación que **combina varios algoritmos** de planificación.

En lugar de existir una sola **Ready Queue**, ésta se divide en varias colas. Cada cola contiene un tipo de procesos y tiene su propio algoritmo de planificación.

Ready Queue:



¿Cómo funciona?

Existen dos planificadores:

1) Planificador Horizontal

Decide **qué proceso ejecutar dentro de una misma cola.**

Ejemplo:

Q0: P1 P2 P3

↓

RR elige:

P1 → P2 → P3 → P1...

Cada cola puede usar un algoritmo distinto:

- Q0 → Round Robin
- Q1 → Prioridades
- Q2 → FIFO

2) Planificador Vertical

Decide **qué cola tiene acceso a la CPU**.

Generalmente:

Q0 Mayor prioridad

↓

Q1

↓

Q2 Menor prioridad

Mientras haya procesos en Q0, no se ejecutan los de Q1.

Mientras haya procesos en Q1, no se ejecutan los de Q2.

Retroalimentación (Feedback)

Un proceso puede cambiar de cola durante su ejecución.

El Sistema Operativo decide cuándo subirlo o bajarlo según el algoritmo utilizado.

Ejemplo 1 (Beneficia CPU Bound)

Queue 0	→ RR Q=8
Queue 1	→ RR Q=16
Queue 2	→ FIFO

Todos los procesos ingresan a cola 0.

Si un proceso **no termina en** los 8 quantums:

baja a la cola 1

Si el proceso no termina en los 16:

baja a la cola 2

En la cola 2 ejecuta por FIFO hasta finalizar.

¿Qué procesos se benefician?

Los **CPU Bound**.

¿Por qué?

Porque necesitan mucha CPU. Entonces bajan hasta Q2 y allí pueden ejecutar ráfagas largas sin ser interrumpidos constantemente.

Ejemplo 2 (Beneficia I/O Bound)

Queue 0	→ RR Q=4
Queue 1	→ RR Q=18
Queue 2	→ FIFO

Todos los procesos ingresan a cola 0.

Si un proceso **agota su** quantum:

baja a la cola 1

Si el proceso **agota su** quantum:

baja a la cola 2

En la cola 2 ejecuta por FIFO hasta finalizar.

Si un proceso abandona la CPU por una operación de E/S antes de consumir su quantum, permanece en la **misma cola** al regresar.

Los procesos de E/S suelen usar muy poca CPU antes de pedir una operación de disco.

Como Q0 tiene la mayor prioridad, siempre vuelve a ejecutarse rápidamente.

¿Qué procesos se benefician?

Los **I/O Bound**.

Porque utilizan poca CPU y realizan muchas operaciones de entrada/salida.

Nunca consumen todo el quantum y permanecen en la cola de mayor prioridad.

Posible desventaja de las colas multinivel

Puede producirse **inanición (starvation)**.

Ejemplo:

Siempre llegan procesos nuevos a Q0.

Como Q0 siempre tiene prioridad, los procesos de Q2 pueden esperar indefinidamente sin ejecutarse.

Memoria

La organización y administración de la memoria RAM es uno de los factores mas importantes en el diseño de un SO.

Los programas y datos deben residir en ella para:

- Poder ejecutar.
- Referenciarlos directamente.

La parte del SO que administra esta memoria se llama “administrador de la memoria”:

- Lleva un registro de las partes de la memoria que se están utilizando y de aquellas que no
- Asigna espacio en memoria a los procesos cuando estos la necesitan.
- Libera espacio de memoria asignada a procesos que han terminado.

Se espera que el SO haga uso eficiente de esta memoria con el fin de alojar el mayor numero de procesos -> repercute en la multiprogramación.

Direccionamiento

Dirección Lógica

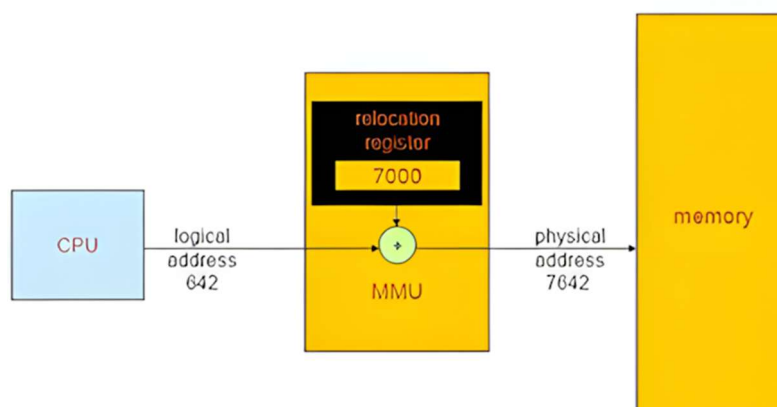
- Es una dirección que enmascara o abstrae una dirección física.
- Referencia a una localidad en memoria.
- Se la debe traducir a una dirección física.

Dirección Física

- Es la dirección real. Es con la que se accede efectivamente a la memoria.
- Representa la dirección absoluta en memoria principal.

La CPU trabaja con direcciones lógicas. Para acceder a la memoria se deben transformar en direcciones físicas.

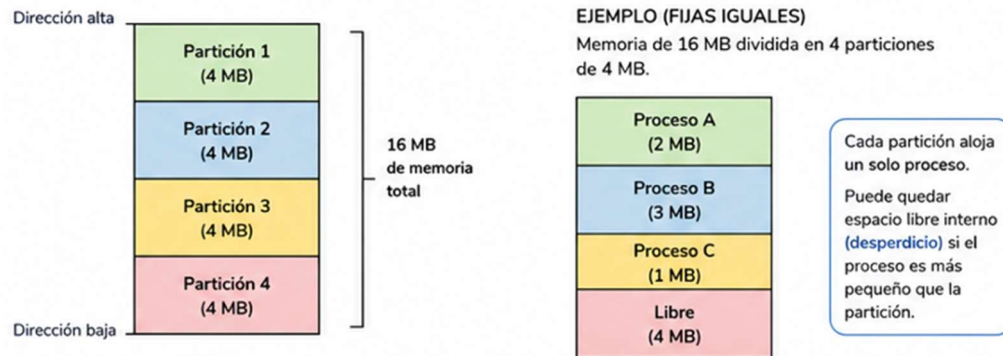
El mapeo entre direcciones virtuales y físicas se realiza mediante Hardware -> **MMU**.



Asignación de memoria

Particiones fijas

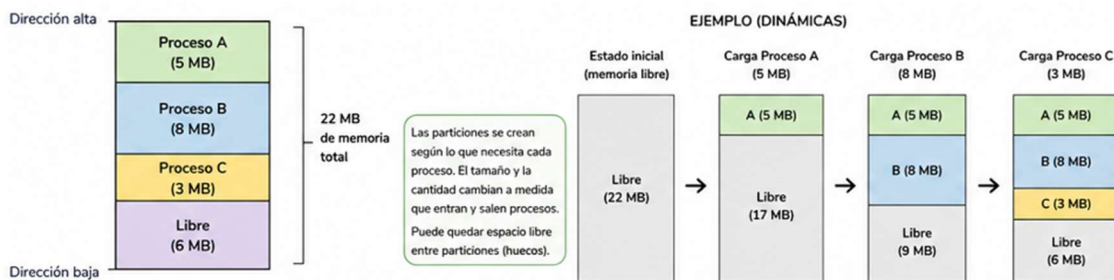
- La memoria se divide en particiones iguales o regiones de tamaño fijo -> tamaños iguales o diferentes.
- Alojan un único proceso.



Ese espacio libre interno que queda en el bloque si el proceso es más pequeño que el tamaño de la partición establecida se denomina -> **Fragmentación Interna**.

Particiones dinámicas

- Las particiones varían en tamaño y número.
- Alojan un proceso cada una.
- Cada partición se genera en forma dinámica del tamaño justo que necesita el proceso



Ese espacio libre interno que queda entre particiones se denomina -> **Fragmentación Externa**.

Selección de partición

Cada proceso se coloca en alguna partición de acuerdo a algún criterio.

- First Fit -> se coloca en la **primera partición libre que sea suficiente**.
- Best Fit -> se coloca en la **partición libre más pequeña que sea suficiente**.
- Worst Fit -> se coloca en la **partición libre más grande que sea suficiente**.
- Next Fit -> Es como First, pero **empieza desde donde termino la última**.

Paginación

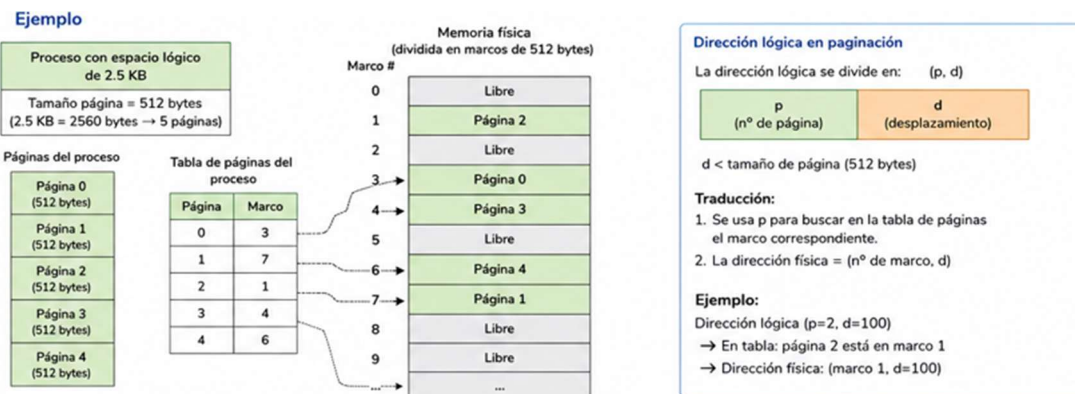
La memoria se divide en porciones de igual tamaño llamadas **marcos**. El espacio de direcciones de los procesos se divide en porciones de igual tamaño denominadas **páginas**.

Tamaño página = tamaño marco = 512 bytes (generalmente).

El SO mantiene **una tabla de páginas para cada proceso**, la cual contiene el marco donde se encuentra cada página.

La paginación bajo demanda es una técnica eficiente de manejar esta estrategia

Paginación -> Particiones fijas -> Fragmentación Interna.



Paginación – Direccionamiento

Un proceso en ejecución hace referencia a una dirección virtual → $v = (p, d)$.

El SO busca la página p en la tabla de páginas del proceso y determina en que marco se encuentra.

La dirección de almacenamiento real se forma por la concatenación de la resolución de p (dirección de inicio del marco que aloca la página) y d, **donde p es el numero de pagina y d es el número de desplazamiento.**

Fórmulas para el Pasaje de direcciones:

• Pasaje de Direccion Logica a Direccion Fisica:

- **Numero de Pagina** = Direccion Logica **DIV** Tamaño de pagina
- **Desplazamiento** = Direccion Logica **DIV** Tamaño de pagina
- **Direccion Fisica** = (Marco * Tamaño de Pagina) + Desplazamiento

• Pasaje de Direccion Fisica a Direccion Logica:

- **Numero de Marco** = Direccion Fisica **DIV** Tamaño de pagina
- **Desplazamiento** = Direccion Fisica **MOD** Tamaño de pagina
- **Direccion Logica** = (Pagina * Tamaño de Pagina) + Desplazamiento

DIV = a / b -> parte entera de la división

MOD = a - (b x parte entera)

Ej:

- 580 DIV 512 = 1,1328 -> 1

- 580 MOD 512 = 580 - (512 * (580 DIV 512)) = 580 - 512 = **68**

Ejemplo de pasajes:

- Memoria administrada por sistema de paginación.

- Tamaño de pagina 512 bytes.

- Cada dirección de memoria referencia a 1 byte.

- Los marco en memoria principal se encuentran desde la dirección física 0.

- Tenemos un proceso con un tamaño de 2000 bytes y con la siguiente tabla de páginas.

Página	Marco		Marco	Inicio-Fin
0	1		0	0 - 511
1	2		1	512 - 1023
2	3		2	1024 - 1535
3	0		3	1536 - 2047

F.L. de 48 B.

Se tiene la siguiente dirección lógica o virtual = **580**

Pasaje a dirección lógica:

Numero de pagina = 580 DIV 512 = 1

Desplazamiento = 580 MOD 512 = 68

Dirección física = (Marco * tamaño de página) + Desplazamiento = 1024 + 68 = **1092**

Pasaje a dirección física:

Siguiendo la lógica si usamos la dirección 1092 como dirección física nos tendría que dar la dirección lógica 580.

Numero de Marco = 1092 DIV 512 = 2

Desplazamiento = 1092 MOD 512 = 68

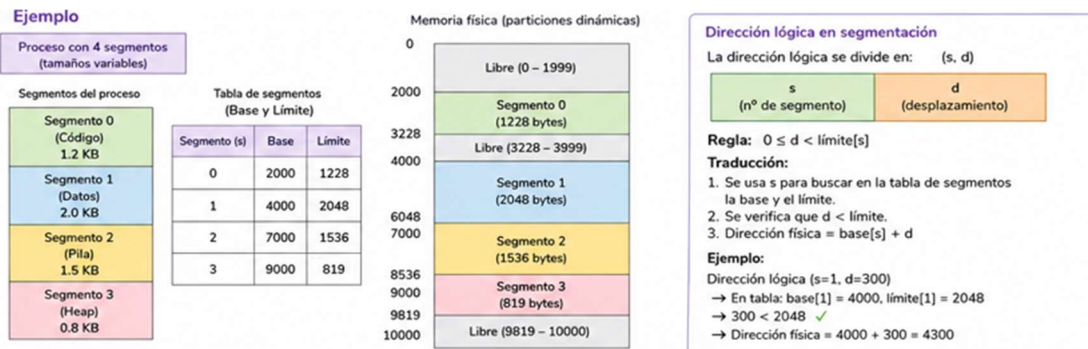
Dirección lógica = (Pagina * tamaño de página) + Desplazamiento = 512 + 68 = **580**

Segmentación

La segmentación se puede ver como una mejora de la paginación, en donde ahora la tabla de segmentos además de tener la dirección de inicio del mismo, tiene la longitud limite.

Las direcciones lógicas constan de dos partes → un numero de segmento s y un desplazamiento d dentro del segmento ($0 < d < \text{limite}$).

Segmentación → Particiones dinámicas → Fragmentación Externa.



Segmentación – Direccionamiento:

Dado una dirección lógica representada de la siguiente manera $(s:d)$ numero de segmento y el desplazamiento $(s:d)$.

- Busco el segmento en la tabla
- Obtengo la **base** y el **tamaño limite**
- Verifico validez: Si el desplazamiento $<$ al tamaño entonces es valido
- Calculo dirección física = **base + desplazamiento**

Ejemplo:

- Dado un esquema de segmentación donde cada dirección hace referencia a 1 byte y la siguiente tabla de segmentos de un proceso, traduzca, de corresponder, las direcciones lógicas indicadas a direcciones físicas. La Dirección lógica está representada por: **segmento:desplazamiento**

Segmento	Dir. Base	Tamaño
0	102	12500
1	28699	24300
2	68010	15855
3	80001	400

-0000:9001 → segmento 0, desplazamiento 9001

Validación: Desplazamiento $<$ tamaño = $9001 < 12500$ ✓

Dirección física = base + desplazamiento = $102 + 9001 = 9103$

-0001:24301 → segmento 1, desplazamiento 24301

Validación: Desplazamiento $<$ tamaño = $24301 > 24300$ ✗

Dirección invalida

Fork

`fork()` es una **llamada al sistema** que crea un **nuevo proceso**

- El proceso que llama a `fork()` se llama **padre**
- El proceso **nuevo** se llama **hijo**
- El hijo es una copia casi exacta del padre (mismo código, variables, etc).

Después de la ejecución del `fork`, los dos procesos se ejecutan concurrentemente.

¿Cómo distinguir quién es el padre y quién es el hijo?

La función devuelve un valor diferente para cada uno:

Hijo -> 0

Padre -> PID del hijo (mayor a 0)

Ejemplo:

```
pid = fork();  
  
if (pid == 0) {  
    // Código del hijo  
}  
else if (pid > 0) {  
    // Código del padre  
}  
else {  
    // Error  
}
```

Cuando hay un `execv` o `exit` o similar el script termina

Esquema de paginación

Formulas:

- Bits para identificar pagina = Tamaño de pagina = $2^n = n$ -> bits p/identificar.
- Tamaño máximo de proceso = $2^{\text{espacio de direcciones}}$
- Max. Paginas por proceso = $\text{Espacio de direcciones} / \text{Tamaño de pagina}$
 $\text{Espacio de direcciones} = 2^{\text{tamaño de dirección}}$
- Cantidad de marcos = $\text{Espacio disponible} / \text{Tamaño de marco}$
 $\text{Tamaño de marco} = \text{Tamaño de pagina}$
- Paginas necesarias = $\text{Tamaño del código o datos} / \text{Tamaño de pagina}$
- Fragmentación interna = $\text{Espacio asignado} - \text{Espacio usado}$
 $\text{Espacio asignado} = \text{Paginas necesarias} * \text{Tamaño de pagina}$
- Cantidad de entradas = $2^{\text{bits para identificar pagina}}$
- Tamaño de mínimo de PTE en RAM = $\text{Cantidad de entradas} * \text{Tamaño PTE}$

Ejemplo:

Dado un esquema de paginación, donde cada dirección hace referencia a 1 byte, se tiene un tamaño de dirección de 32 bits y un tamaño de página de 2048 bytes. Suponga además un proceso P1 que necesita 51358 bytes para su código y 68131 bytes para sus datos.

- Cada dirección referencia a 1 byte
- Tamaño de dirección 32 bits $\rightarrow$ 4 bytes
- Tamaño de página de 2048 bytes $\rightarrow 2^{11}$ bytes
- P1 necesita 51358 bytes para su código y 68131 bytes para sus datos.
- Espacio de direcciones = $2^{\text{Tamaño de dirección}}$ $\rightarrow 2^{32}$ bytes

a. De los 32 bits de las direcciones ¿Cuántos se utilizan para identificar página?
¿Cuántos para desplazamiento?

Una página tiene un tamaño de 2048 bytes $\rightarrow 2^{11}$ bytes y cada dirección tiene 32 bits
 $2^{11} \rightarrow 11$ bits para el desplazamiento
El resto para el número de página $\rightarrow 32 - 11 = 21$ bits para el número de página

b. ¿Cuál sería el tamaño lógico máximo de un proceso?

Tamaño máximo de un proceso = **$2^{\text{Espacio de direcciones}}$**
 $= 2^{32}$ bytes

c. ¿Cuántas páginas máximo puede tener el proceso P1?

Cantidad máxima de páginas por proceso = **$\text{Espacio de direcciones} / \text{Tamaño de páginas}$**
 $= 2^{32} / 2^{11} = 2^{21}$ bytes

d. Si se contaran con 4GiB (Gibibytes) de RAM, ¿cuántos marcos habría disponibles para utilizar?

Ram de 4GiB $\rightarrow 1 \text{ GiB} = 2^{30}$ bytes $\rightarrow 4 \text{ GiB} = 2^{30} \times 4 = 2^{30} \times 2^2 = 2^{32}$ bytes
Tamaño de marco = Tamaño de página = 2^{11} bytes
Cantidad de marcos = **$\text{Espacio disponible} / \text{Tamaño de marco}$**
 $= 2^{32} / 2^{11} = 2^{21}$ bytes

e. ¿Cuántas páginas necesita P1 para almacenar su código?

Páginas necesarias = **$\text{Tamaño del código} / \text{Tamaño de página}$**
 $= 51358 / 2048 = 25,07 = 26 \rightarrow$ siempre redondeo para arriba

f. ¿Cuántas páginas necesita P1 para almacenar sus datos?

Paginas necesarias = **Tamaño de los datos / Tamaño de pagina**
 = $38131 / 2048 = 33,26 = 34 \rightarrow$ **siempre redondeo para arriba**

g. ¿Cuántos bytes habría de fragmentación interna entre lo necesario para almacenar su código y sus datos? Ayuda: Recuerde que en una misma página no pueden convivir código y datos.

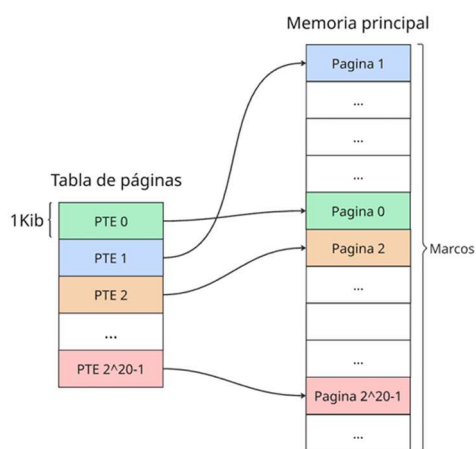
código = 51358 bytes datos = 68131 bytes

Fragmentación interna = **espacio asignado – espacio usado =**
 = $53248 - 51358 = 1890$ **bytes de fragmentación interna para el código de P1**
 espacio asignado = **Paginas necesarias * Tamaño de pagina = 26×2048**
 = 53248

Fragmentación interna = $34 \times 2048 - 68131 = 69632 - 68131$
 = **1501 bytes de fragmentación interna para el código de P1**

Fragmentación interna total = $1890 + 1501 = 3391$ bytes

h. Tabla de páginas de 1 nivel osea:



En donde:

- 20 bits para identificar página
- 12 bits para identificar desplazamiento
- PTE de 1 KiB $\rightarrow 2^{10}$ bytes

Buscamos saber el tamaño mínimo de la tabla de páginas del P1

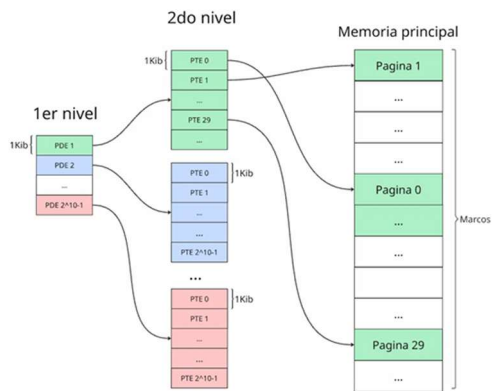
Cantidad de paginas posibles = $2^{\text{bits para identificar pagina}}$
 = 2^{20}

1 nivel $\rightarrow 2^{20}$ páginas $\rightarrow 2^{20}$ entradas

Entonces 2^{20} entradas x Tamaño de PTE

= $2^{20} \times 2^{10} = 2^{30}$ **bytes**

i. Tabla de páginas de 2 niveles osea:



10 bits para primer nivel $\rightarrow 2^{10}$ entradas
 10 bits para segundo nivel $\rightarrow 2^{10}$ entradas
 12 bits para desplazamiento $\rightarrow 2^{12}$ bytes
 $\rightarrow$ tamaño de página
 PTE 1Kib $\rightarrow 2^{10}$ bytes

-Cantidad de páginas de P1 para código = **13**

tamaño código / tamaño de página = $51358 / 2^{12} = 12,53 \rightarrow 13$

-Cantidad de paginas de P1 para datos = **17**

tamaño datos / tamaño de pagina = $68131 / 2^{12} = 15,63 \rightarrow 17$

Primer nivel de la tabla tiene 2^{10} entradas y cada PTE requiere 2^{10} bytes
 $= 2^{20}$ bytes

Segundo nivel de la tabla tiene 2^{10} entradas y cada PTE requiere 2^{10} bytes
 $= 2^{20}$ bytes

Entonces como tenemos que almacenar 30 páginas (13 + 17) con una sola tabla del segundo nivel nos va a alcanzar para almacenar las páginas del proceso 1. Por lo tanto como mínimo va a ocupar:

Tabla del primer nivel + Tabla del segundo nivel = $2^{20} + 2^{20} = 2 \times 2^{20} \rightarrow 2^2 \times 2^{20}$
 $= 2^{21}$ bytes

i. - **Tabla de un nivel** $\rightarrow$ ineficiente en cuanto espacio

- **Tabla de dos niveles** $\rightarrow$ más compleja de mantener, pero ahorra espacio. Además, requiere más accesos a memoria para poder resolver una dirección.